

Comunicación entre Procesos

Memoria compartida, semáforos, mmap, pipes y sockets



```
$ gcc -o shm shm.c
$ ./shm
adjuntado en 0x7f3a2c000000
Hola, mundo.
```

Contenido

1

Memoria compartida

shmget, shmat, shmdt — la IPC más rápida

2

Semáforos de proceso

semget, semop — coordinar el acceso compartido

3

Memoria mapeada

mmap, munmap — memoria compartida atada a un archivo

4

Pipes y FIFOs

pipe, dup2, popen, mkfifo — comunicación en un sentido

5

Sockets

socket, bind, listen, accept — entre procesos y por red

INTRODUCCIÓN

¿Por qué no alcanza con `fork()`/`wait()`?

- En el Capítulo 3 se mostró cómo crear procesos y esperarlos, pero estos mecanismos solo permiten la comunicación a través del código de salida.
- Ningún mecanismo de `fork()`/`wait()` permite hablar con un proceso mientras corre, ni con uno que no sea hijo propio.
- Este capítulo cubre 5 mecanismos que resuelven eso — cada uno con sus propias reglas de juego.

Procesos emparentados

- Memoria compartida y memoria mapeada: leen y escriben la misma memoria directamente.
- Pipes: el hijo hereda los descriptores del padre al hacer `fork()`.
- Las formas más simples y rápidas — pero exigen parentesco.

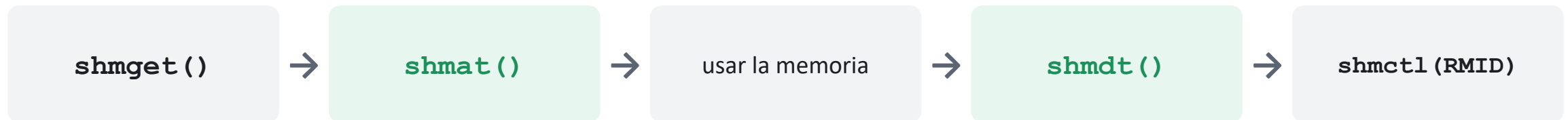
Cualquier proceso, o en red

- FIFOs: un pipe con nombre en el filesystem, visible para cualquier proceso.
- Sockets: la única opción para comunicarse entre computadoras distintas.
- Más flexibles, a costa de un poco más de configuración.

MEMORIA COMPARTIDA

La forma más rápida de comunicarse

- Varios procesos puede acceder a la misma memoria física, como si todos hubieran llamado a `malloc()` y recibido el mismo puntero.
- Es el mecanismo de IPC más rápida: no hay llamada al sistema de por medio en cada acceso, solo lectura y escritura directa.
- El kernel NO sincroniza los accesos: si dos procesos escriben a la vez, el resultado es indefinido. (De ahí el próximo tema: semáforos.)



MEMORIA COMPARTIDA

Código – shm.c

```
// gcc -o shm shm.c
#include <stdio.h>
#include <sys/shm.h>
#include <sys/stat.h>
int main(void) {
    int segment_id;
    char* shared_memory;
    const int size = 0x6400;
    // 1. Asignar el segmento.
    segment_id = shmget(IPC_PRIVATE, size,
                       IPC_CREAT | IPC_EXCL | S_IRUSR | S_IWUSR);
    // 2. Adjuntarlo al espacio de direcciones del proceso.
    shared_memory = (char*) shmat(segment_id, 0, 0);
    printf("adjuntado en %p\n", shared_memory);
    // 3. Escribir en la memoria compartida.
    sprintf(shared_memory, "Hola, mundo.");
    shmdt(shared_memory);
    // 4. Otro proceso podria adjuntarse y leer el string aqui.
    shared_memory = (char*) shmat(segment_id, 0, 0);
    printf("%s\n", shared_memory);
    shmdt(shared_memory);
    // 5. Liberar el segmento cuando nadie mas lo necesita.
    shmctl(segment_id, IPC_RMID, 0);
    return 0;
}
```

SEMÁFOROS DE PROCESO

Coordinar el acceso compartido

- Son la versión "entre procesos" de los semáforos.
- `semop()` implementa espera y aviso a la vez: un número negativo resta (y bloquea si hace falta), uno positivo suma.

```
semget() / semctl()
```

Asignan y liberan el conjunto de semáforos — como `shmget/shmctl` para memoria compartida.

```
semop() con sem_op < 0
```

Espera: bloquea hasta que el valor sea suficiente, y lo decrementa.

```
semop() con sem_op > 0
```

Avisa: incrementa el valor. Nunca bloquea al proceso que llama.

SEMÁFOROS DE PROCESO

Código – sem_pv.c

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
// Espera: bloquea hasta que el valor sea positivo, y lo decreuenta.
int binary_semaphore_wait(int semid) {
    struct sembuf op[1];
    op[0].sem_num = 0;
    op[0].sem_op = -1;          // decrementar en 1
    op[0].sem_flg = SEM_UNDO;
    return semop(semid, op, 1);
}
// Aviso: incrementa el valor en 1. Nunca bloquea.
int binary_semaphore_post(int semid) {
    struct sembuf op[1];
    op[0].sem_num = 0;
    op[0].sem_op = 1;          // incrementar en 1
    op[0].sem_flg = SEM_UNDO;
    return semop(semid, op, 1);
}
```

MEMORIA MAPEADA

Memoria compartida atada a un archivo

- `mmap()` asocia un archivo con una región de memoria del proceso: al leer y escribir esa memoria, se lee y escribe el archivo.
- `munmap()` libera el mapeo.

MAP_SHARED

- Los cambios se escriben en el archivo y los ve cualquier otro proceso que lo mapee.
- Es el modo que se usa para IPC.
- `msync()` fuerza el volcado a disco antes de lo habitual.

MAP_PRIVATE

- Copy-on-write: los cambios quedan solo en la copia del proceso.
- Ningún otro proceso ve las modificaciones.
- Útil para leer (o modificar sin guardar) un archivo grande sin pagar el costo de `read()`.

MEMORIA MAPEADA

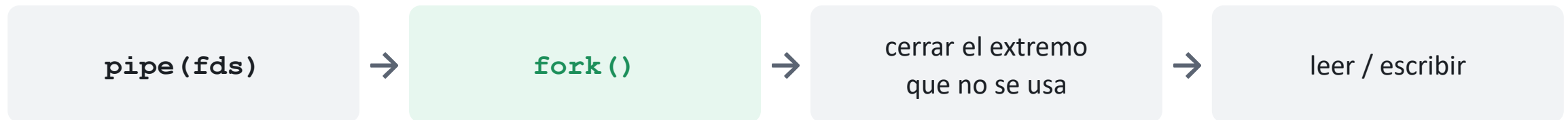
Código – mmap_write.c

```
// gcc -o mmap_write mmap_write.c
#include <stdio.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <unistd.h>
#define TAM 0x100
int main(int argc, char** argv) {
    int fd;
    void* memoria;
    // Crear el archivo, del tamaño necesario.
    fd = open(argv[1], O_RDWR | O_CREAT, S_IRUSR | S_IWUSR);
    lseek(fd, TAM, SEEK_SET);
    write(fd, "", 1);
    // Mapear el archivo a memoria.
    memoria = mmap(0, TAM, PROT_WRITE, MAP_SHARED, fd, 0);
    close(fd); // el mapeo sigue vivo aunque cerremos el fd
    // Escribir directo en memoria: el archivo cambia solo.
    sprintf((char*) memoria, "%d\n", 42);
    munmap(memoria, TAM);
    return 0;
}
```

PIPES Y FIFOS

Comunicación en un solo sentido

- `pipe()` crea dos descriptors: `fds[0]` para leer, `fds[1]` para escribir. Lo escrito de un lado sale, en orden, del otro.
- Solo conecta procesos emparentados: `fork()` copia los descriptors al hijo, así padre e hijo terminan compartiendo el mismo pipe.
- Si quien escribe es más rápido que quien lee (o al revés), el pipe bloquea solo — no hace falta sincronizar a mano.



PIPES Y FIFOS

Código – pipe_demo.c

```
// gcc -o pipe_demo pipe_demo.c
#include <stdio.h>
#include <unistd.h>
void escritor(const char* msg, int n, FILE* out) {
    for (; n > 0; --n) { fprintf(out, "%s\n", msg); fflush(out); sleep(1); }
}
void lector(FILE* in) {
    char buffer[1024];
    while (fgets(buffer, sizeof(buffer), in) != NULL)
        fputs(buffer, stdout);
}
int main(void) {
    int fds[2];
    pipe(fds);           // fds[0]=lectura, fds[1]=escritura
    if (fork() == 0) {
        close(fds[1]);    // el hijo no escribe
        lector(fdopen(fds[0], "r"));
    } else {
        close(fds[0]);    // el padre no lee
        escritor("Hola, mundo.", 5, fdopen(fds[1], "w"));
    }
    return 0;
}
```

PIPES Y FIFOS

Redirigir un pipe: `dup2()` y el atajo `popen()`

- `dup2(fd, STDIN_FILENO)` hace que un descriptor "sea" la entrada estándar — así un hijo puede leer del pipe como si fuera el teclado.
- Es el mecanismo detrás de comandos como `ls | sort`: el hijo redirige su entrada al extremo de lectura antes de hacer `exec()`.
- `popen()/pclose()` empaquetan `pipe()+fork()+dup2()+exec()` en una sola llamada, a costa de perder algo de control fino.

```
dup2(fd, STDIN_FILENO)
```

El hijo lee del pipe como si fuera su entrada estándar, antes de `exec()`.

```
popen("sort", "w")
```

Atajo: crea el proceso y el pipe en un solo paso; `pclose()` lo espera.

PIPES Y FIFOS

FIFOs: pipes con nombre en el filesystem

- Un FIFO (o pipe con nombre) es un pipe que aparece como un archivo — se crea con `mkfifo()`.
- A diferencia de un pipe común, dos procesos SIN relación de parentesco pueden comunicarse a través de él.
- Se abre y se usa como cualquier archivo: con `open()/read()/write()` o con `fopen()/fprintf()/fscanf()`.

```
$ mkfifo /tmp/fifo
$ ls -l /tmp/fifo
prw-rw-rw- 1 user users 0 ene 16 14:04 /tmp/fifo
$ cat < /tmp/fifo          # ventana 1: espera datos...
$ cat > /tmp/fifo          # ventana 2: escribe y Ctrl+D
Hola, mundo.
```

SOCKETS

Comunicación en red

- Un socket es un dispositivo de comunicación bidireccional — entre procesos en la misma máquina, o en máquinas distintas.
- Namespace: local (PF_LOCAL, direcciones = nombres de archivo) o de Internet (PF_INET, direcciones = IP + puerto).

SOCK_STREAM

- Garantiza orden y entrega de todos los paquetes.
- Como una llamada telefónica: la conexión se establece una vez, al principio.

SOCK_DGRAM

- No garantiza ni orden ni entrega — "mejor esfuerzo".
- Como el correo postal: cada mensaje lleva su propia dirección de destino.

SOCKETS

El ciclo de vida de un servidor

- Todo servidor sigue el mismo patrón, sin importar qué datos transporte:



→ Un socket nuevo por cliente

`accept()` no reutiliza el socket original: crea uno nuevo para hablar con ese cliente, mientras el primero sigue escuchando otras conexiones.

SECCIÓN 5 · SOCKETS

Código – srv.c (servidor local)

```
// gcc -o srv srv.c
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <unistd.h>
int main(int argc, char** argv) {
    int fd = socket(PF_LOCAL, SOCK_STREAM, 0), cfd, n;
    char* t;
    struct sockaddr_un nom = { .sun_family = AF_LOCAL };
    strcpy(nom.sun_path, argv[1]);
    bind(fd, (struct sockaddr*) &nom, SUN_LEN(&nom));
    listen(fd, 5);
    while (1) {
        cfd = accept(fd, NULL, NULL);
        while (read(cfd, &n, sizeof(n)) > 0) {
            t = malloc(n);
            read(cfd, t, n);
            printf("%s\n", t);
            free(t);
        }
        close(cfd);
    }
}
```

SECCIÓN 5 · SOCKETS

Código – cli.c (cliente local)

```
// gcc -o cli cli.c
#include <string.h>
#include <sys/socket.h>
#include <sys/un.h>
#include <unistd.h>

void enviar(int fd, const char* texto) {
    int n = strlen(texto) + 1;
    write(fd, &n, sizeof(n)); // primero, el largo
    write(fd, texto, n);      // despues, el texto
}

int main(int argc, char** argv) {
    int fd = socket(PF_LOCAL, SOCK_STREAM, 0);
    struct sockaddr_un nom = { .sun_family = AF_LOCAL };
    strcpy(nom.sun_path, argv[1]);
    connect(fd, (struct sockaddr*) &nom, SUN_LEN(&nom));
    enviar(fd, argv[2]);
    close(fd);
    return 0;
}
```

SECCIÓN 5 · SOCKETS

Sockets de Internet: la misma idea, otra escala

- PF_INET conecta procesos en máquinas distintas — el mismo socket(), pero otro namespace.
- La dirección combina una IP (struct sockaddr_in, campo sin_addr) y un puerto (sin_port, convertido con htons()).
- gethostbyname() traduce un nombre de dominio ("www.ejemplo.com") a una dirección IP.
- Los servicios conocidos usan puertos fijos: 80 para HTTP, 443 para HTTPS, 25 para SMTP.

```
struct sockaddr_in dir = { .sin_family = AF_INET };
dir.sin_addr = *((struct in_addr*) gethostbyname(host)->h_addr);
dir.sin_port = htons(80);
connect(fd, (struct sockaddr*) &dir, sizeof(dir));
```

RESUMEN

5 mecanismos

Memoria compartida

La más rápida, a través de la memoria del Kernel.

Memoria mapeada

Memoria compartida vinculada a un archivo.

Pipes / FIFOs

Simple y ordenado, en un solo sentido.

Sockets

La única opción entre máquinas distintas.

05

FIN DEL CAPÍTULO 5

¿Preguntas?